

Cet examen est composé de 3 exercices indépendants qui peuvent être traités dans un ordre quelconque. Il est donc conseillé de lire l'énoncé dans sa totalité avant de commencer.

Pour les questions commençant par *“Proposer une solution”* vous devez expliquer textuellement comment fonctionne votre solution, pour les questions commençant par *“Donner l'algorithme”* vous devez donner l'algorithme, avec des commentaires.

Pour les algorithmes, vous ne disposez pas d'autres objets/fonctions que des listes, files ou piles et leurs fonctions associées définies dans le cours.

Attention, la note sera diminuée si :

- Le soin apporté à la rédaction et à la copie sont insuffisants,
- Toutes les réponses ne sont pas justifiées par un raisonnement clair et précis.

Exercice 1 : Utilisation d'un pédalo (5 points)

Au bord d'un lac, un camping offre à ses clients la possibilité d'utiliser un pédalo, à condition de le réserver par mail le jour précédent, en précisant l'heure p à laquelle le demandeur veut prendre le pédalo et l'heure r à laquelle il le rend. Comme les clients ne se concertent pas, il peut y avoir des demandes qui se recouvrent.

Chaque jour, à partir de ces demandes, la direction du camping doit établir un planning d'attribution du pédalo, dans lequel elle souhaite qu'un maximum de ses clients soient satisfaits.

Question 1.1 : *Proposer une solution gloutonne calculant ce planning*

Question 1.2 : *Donner l'algorithme d'une fonction qui implémente cette proposition.*

Correction Exercice 1 : Utilisation d'un pédalo

Solution question 1.1 : *Proposer une solution gloutonne calculant ce planning*

Ce problème est équivalent au problème du choix des activités du cours. Il suffit donc de l'adapter au contexte. Ici il faut trier les demandes par leur dates de terminaison au plus tôt et choisir, de manière gloutonne, les demandes compatibles en commençant par le début.

Solution question 1.2 : *Donner l'algorithme qui implémente cette proposition.*

L'algorithme 1 donne une solution récursive au problème du choix des demandes. Cet algorithme est optimal. Vous pouvez relire la partie “Problème du choix d'activités” du cours pour les explications. plus d'explications.

Algorithme 1 : Algorithme itératif glouton choisissant un nombre optimal d'activités mutuellement compatibles parmi un ensemble d'activités ordonnées

```

/* Les listes sont triées par ordre croissant des dates de fin          */
1  p : liste des dates de début des demandes
2  r : liste des dates de fin des demandes
3  Fonction selectionDemadesGlouton(p, r)
4  début
5      n ← r.size()
6      D.add(r.get(0)) /* La première demande (0) est toujours choisie */
7      i ← 0
      /* Recherche la première demande compatible avec di          */
8      pour m de 1 à n − 1 faire
9          si r.get(i) ≤ p.get(m) alors
10             D.add(m)
11             i ← m
12  retourner A

```

Exercice 2 : Jeu vidéo (8 points)

Dans un jeu vidéo un vaisseau spatial doit détruire une base distante en lui envoyant des ondes négatives. Plus le volume d'ondes envoyé est important, plus le vaisseau a de chance de détruire la base.

Pour ne pas se mettre en danger le vaisseau reste à distance de la base et utilise, comme relais, un ensemble de satellites qu'il a piraté. La portée et la capacité de relais des satellites sont limitées, ainsi le vaisseau doit émettre ses ondes à destination des satellites les plus proches qui relaient vers d'autres satellites, eux même relayant, jusqu'à arriver aux satellites à portée de la base. La figure 1 donne des connexions possibles entre des satellites, un vaisseau et une base.

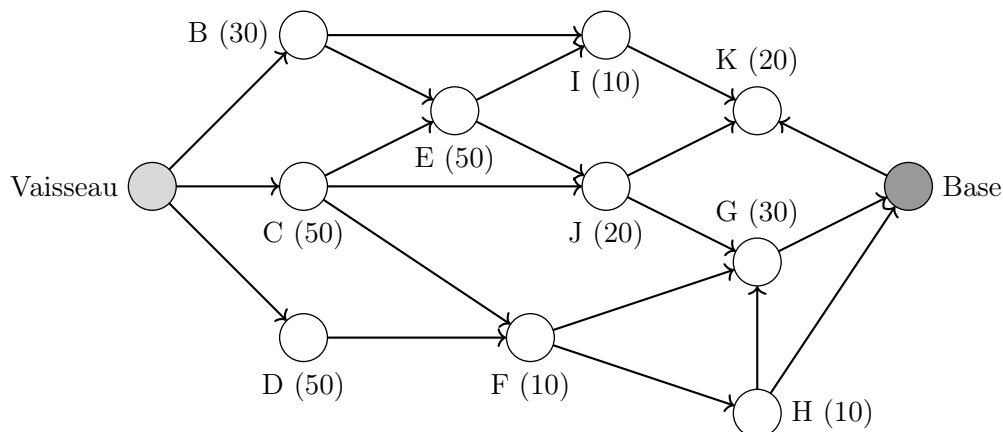


FIGURE 1 – Exemple de graphe de satellites entre un vaisseau et une base. Les flèches représentent les satellites accessibles, les valeurs numériques sont les capacités de relais

Émettre des ondes est néanmoins coûteux pour le vaisseau et, pour ne pas les gaspiller, il a besoin de savoir quel volume d'ondes au maximum il peut émettre vers chacun des satellites qu'il peut atteindre en étant sûr que toutes ces ondes arrivent à la base.

Question 2.1 : *Donner le résultat pour le graphe donné à la figure 1. Expliquer comment vous réalisez ce calcul.*

Question 2.2 : *Expliquer comment procéder pour calculer le volume maximal qui peut être envoyé du vaisseau vers la base.*

Nous supposons que le joueur dispose d'une matrice d'adjacence S de dimension N , dans laquelle le vaisseau est le sommet 0, la base le sommet $N - 1$ et les satellites les autres sommets. Les capacités de relais des satellites sont données sur la diagonale de la matrice. Les arcs du graphe donnent les satellites accessibles depuis un sommet.

Question 2.3 : *Donner une fonction qui rend un chemin entre le vaisseau et la base, sous la forme d'une liste et tel que les sommets du chemin ont tous une capacité de relais non nulle*

Question 2.4 : *Donner l'algorithme général rendant un tableau des voisins du vaisseau et le volume qui peut être envoyé à chacun des voisins.*

Correction Exercice 2 : Jeu vidéo

Le problème posé par cet exercice s'apparente à un problème de max flow. Il diffère du problème classique, pour lequel les limitations sont données sur les arêtes, par des limitations qui sont données sur les sommets. Néanmoins, nous voyons que la même stratégie peut s'appliquer ici : prend un chemin, utiliser sa capacité max, puis chercher les chemins suivants en tenant compte de ce qui est réservé précédemment.

Solution question 2.1 : *Donner le résultat pour le graphe donné à la figure 1 . Pour réaliser le calcul nous prenons le premier*

Avec le graphe donné à la figure 1, le vaisseau peut envoyer un volume d'onde maximal de 60. Pour faire ce calcul, nous prenons les chemins existants entre le vaisseau et la base. Pour chaque chemin nous regardons le volume qu'il est possible de faire passer. Ce volume est retiré des satellites relais avant de recommencer. Ici nous prenons un premier chemin $\{Vaisseau, B, I, K\}$ et nous regardons le volume d'ondes qu'il est possible de faire passer : le sommet limitant est le sommet I dont la capacité maximale est de 10. Nous réservons donc 10 sur les sommets du chemin : la capacité de B passe à 20, celle de I à 0 et celle de K à 10. Nous partons ensuite du vaisseau. Il n'y a plus de chemin possible utilisant l'arc de B à I , par contre, depuis B nous pouvons passer par E, J puis G pour envoyer un volume de 20, le chemin étant limité par B et G . Pour finir, le vaisseau peut encore utiliser le chemin $\{Vaisseau, C, F, H\}$, limité à 10.

Solution question 2.2 : *Expliquer comment procéder pour calculer le volume maximal qui peut être envoyé du*

L'idée générale est donc d'abord de trouver des chemins entre le vaisseau et la base. Lorsqu'un chemin est trouvé, il faut chercher le volume maximal qui peut y passer. Ce volume maximal est affecté au premier voisin du vaisseau et retiré de tous les satellites du chemin. Lorsque le volume de relais d'un satellite tombe à 0, celui-ci ne peut plus être utilisé par les chemins ses liens sont donc retirés de la matrice.

La recherche s'arrête lorsque l'algorithme de parcours ne trouve plus de chemin.

Pour mémoriser les volumes par voisins du vaisseau, nous utilisons une liste utilisant les mêmes indices que la matrice.

Question 2.5 : *Donner une fonction qui rend un chemin entre le vaisseau et la base, sous la*

forme d'une liste et tel que les sommets du chemin ont tous une capacité de relais non nulle

La fonction donnée par l'algorithme 2 rend un chemin allant du vaisseau à la base, sur lequel la capacité de relais des satellites est non nulle. Comme le graphe est orienté il est suffisant de parcourir la partie supérieure de la matrice. Comme dans l'algorithme du cours, nous utilisons une pile pour empiler les sommets voisins et on prend le premier sommet de la pile pour continuer la recherche et donc faire une recherche en profondeur. Cependant, à la différence du parcours nous nous arrêtons dès que le sommet $N - 1$, représentant la base, est trouvé.

Algorithme 2 : Parcours en profondeur du graphe S pour la recherche d'un chemin vers la base

Entrées : S : matrice de dimension N

Sorties : C : liste de sommets reliant le vaisseau à la base

1 **Fonction** *cheminBase*(S) :

Données : p : pile des sommets en cours, **init à** *pileVide*()

trouve, *prochainVoisin* : booléen

2 **début**

3 $p.push(0)$

4 *trouveBase* $\leftarrow$ *faux*

5 **tant que** $\neg p.empty() \wedge \neg trouveBase$ **faire**

6 $u \leftarrow p.peek()$

7 $C.add(u)$

8 **si** $u = N - 1$ **alors** *trouveBase* $\leftarrow$ *vrai*

9 **sinon**

10 $v \leftarrow u + 1$

11 *trouve* $\leftarrow$ *faux*

12 **tant que** $(\neg trouve) \wedge (v \leq N - 1)$ **faire**

13 **si** $S[u, v] = 1 \wedge S[v, v] > 0$ **alors**

14 $p.push(v)$

15 *trouve* $\leftarrow$ *vrai*

16 $v \leftarrow v + 1$

17 **si** $\neg trouve$ **alors**

18 $C.remove()$

19 $p.pop()$

20 **retourner** C

Solution question 2.3 : Donner l'algorithme général

Nous adaptons l'algorithme de Ford-Fulkerson pour proposer une solution dans l'algorithme 3. L'algorithme recherche des chemins sur lesquels il y a encore du volume disponible. Lorsqu'un chemin est trouvé, il calcule le maximum de volume qui peut être émis dans cette direction, en cherchant la plus petite valeur de volume disponible le long du chemin. Cette valeur est ensuite déduite de tous les sommets du chemin et ajoutée dans la liste des voisins. L'algorithme s'arrête lorsqu'il n'y a plus de chemin.

Algorithme 3 : Recherche $G(V, E)$ à partir du sommet s

Entrées : S : matrice de dimension N **Sorties** : V : liste de couples (sommet,volume) /* pour chq voisin de la base */**1 Fonction** *volumeMax*(S) :**Données** : C : liste $trouve$: booléen $premSat, minVol$: entiers**2 début****3** $C \leftarrow cheminBase(S)$ **4** **tant que** $C.size() > 0$ **faire**

/* Recherche du vol max sur le chemin */

5 $premSat \leftarrow C.get(1)$ **6** $minVol \leftarrow S[premSat, premSat]$ **7** **for** $i \leftarrow 2$ à $C.size() - 1$ **do****8** **si** $S[i, i] < minVol$ **alors** $minVol \leftarrow S[i, i]$

/* Ajout du volume au sattelite voisin */

9 $trouve \leftarrow faux$ **10** $i \leftarrow 0$ **11** **tant que** $(i < V.size() - 1) \wedge \neg trouve$ **faire****12** **si** $V.get(i).sommet = premSat$ **alors****13** $trouve = vrai$ **14** $V.get(i).volume \leftarrow V.get(i).volume + minVol$ **15** $i \leftarrow i + 1$ **16** **si** $\neg trouve$ **alors** $R.add(premSat, minVol)$

/* Suppression du volume sur le chemin */

17 **for** $i \leftarrow 1$ à $C.size() - 1$ **do** $S[i, i] \leftarrow S[i, i] - minRel$ **18** $C \leftarrow cheminBase(S)$ **19** **retourner** R

Exercice 3 : Partage de trésor (7 points)

Deux chercheurs de trésor ont trouvé un coffre contenant de pièces d'or, de valeurs différentes. Ils s'interrogent sur la meilleure manière de se partager équitablement ce trésor, pour que chacun ait la même somme. Ils ont alors le dialogue suivant :

- *Premier chercheur* : ce n'est pas forcément possible ! Si la somme totale est impaire, il n'y a pas de solution... et même si la somme est paire, certains cas n'ont pas de solution. Par exemple, si le trésor est constitué de 4 pièces de valeur 4 et d'une pièce de valeur 6, alors nous ne pourrions pas partager de manière équitable.

- *Second chercheur* : Effectivement mais il y a aussi des cas où il y a une solution. Commençons par compter l'ensemble des pièces pour en obtenir la valeur totale N . Si nous pouvons faire un ensemble de pièces de valeur $N/2$ alors nous avons un partage équitable. Pour réaliser cet ensemble nous pouvons étudier, pour chaque pièce, les deux cas suivants : celui où nous l'ajoutons à l'ensemble que nous voulons faire et celui où ne l'ajoutons pas. Dans les deux cas le nombre de pièces à traiter est plus petit et une fois que nous avons traité toutes les pièces, nous avons la réponse.

Question 3.1 : Appliquer la méthode proposée par le second chercheur à l'ensemble $P = \{2, 4, 6, 8\}$. Existe-t-il une solution ?

Question 3.2 : Proposer une formulation récursive de la fonction partage qui rend vrai ou faux suivant qu'elle a trouvé une solution au problème dans le cas général où n est le nombre de pièces du trésor, $P = \{p_1, p_2, \dots, p_n\}$ l'ensemble des pièces, T la valeur totale du trésor et $\text{int val}(p)$ une fonction qui donne la valeur d'une pièce p .

Question 3.3 : Proposer une fonction `partageDyn` utilisant la programmation dynamique, prenant en entrée l'ensemble P , sous la forme d'une liste de valeurs des pièces, et rendant vrai si l'ensemble est partageable et faux sinon

Vous pouvez utiliser les valeurs T , n et la fonction `val` définies à la question précédente.

Question 3.4 : Comment modifier la fonction `partageB` pour obtenir une liste des pièces qui représente la moitié du trésor ?

Correction Exercice 3 : Partage de trésor

Solution question 3.1 : Appliquer la méthode proposée par le second chercheur à l'ensemble $P = \{2, 4, 6, 8\}$. Existe-t-il une solution ?

Avec la méthode proposée, il faut commencer par calculer la somme totale, ici 10. Cette somme est paire et la moitié représente 5, qui est donc notre objectif. Si nous étudions les pièces une à une, nous obtenons l'arbre donné à la figure 2. Il est bien possible de couper l'ensemble P en deux parts égales : il existe deux solutions qui sont données en vert sur la figure.

Sur cette figure nous pouvons constater l'explosion combinatoire engendrée par l'exploration exhaustive des solutions : pour un ensemble de 4 pièces nous avons 16 (2^4) combinaisons. Le problème est en fait un problème classique, appelé problème de 2 partitions, qui est NP-Complet.

Solution question 3.2 : Proposer une formulation récursive de la fonction partage qui rend vrai ou faux suivant qu'elle a trouvé une solution au problème dans le cas général où n est le nombre de pièces du trésor, $P = \{p_1, p_2, \dots, p_n\}$ l'ensemble des pièces, T la valeur totale du trésor, `val(p)` une fonction qui donne la valeur d'une pièce p .

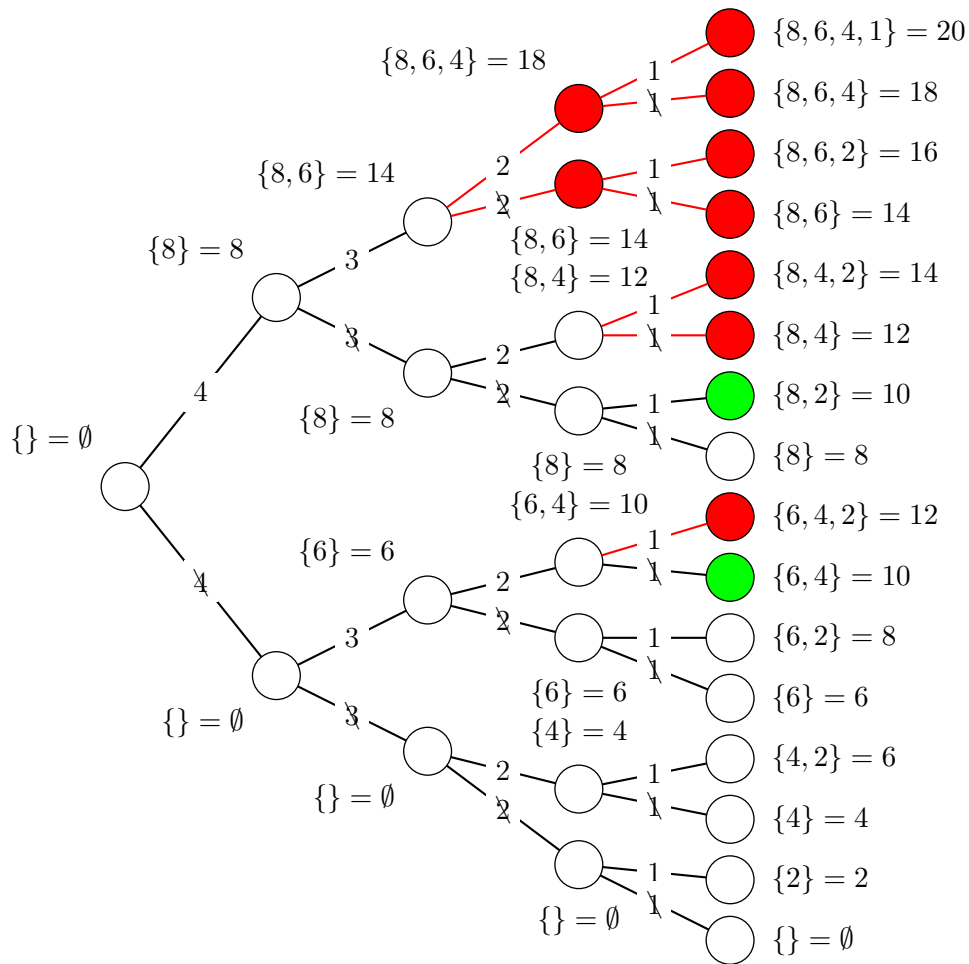


FIGURE 2 – Représentation sous la forme d'un arbre des solutions au problème de partition de l'ensemble P

Comme l'a souligné le second chercheur, il est possible de considérer chacune des pièces à tour de rôle pour obtenir une formulation récursive.

$$partage(P, n, T/2) = \begin{cases} \text{vrai si } P = \emptyset, \\ partage(P - \{p_i\}, n - 1, T/2) \vee partage(P - \{p_1\}, n - 1, T/2 - val(p_1)) \end{cases}$$

Comment nous pouvons le voir sur la figure 2 cette formulation du problème conduit à une exploration de branches (en rouge sur la figure) qui ne donnent pas de solution, car la somme visée est dépassée.

Solution question 3.3 : Proposer une fonction de programmation dynamique *partage*, prenant en entrée l'ensemble P , sous la forme d'une liste de valeurs des pièces, et rendant vrai si l'ensemble est partageable et faux sinon

L'algorithme 4 utilise la programmation dynamique car il s'appuie sur les solutions précédemment calculées pour trouver les solutions des plus grands problèmes. Pour chacune des valeurs de pièces, nous mettons à jour le tableau

Contrairement à plusieurs problèmes résolus par programmation dynamique vus dans le cours,

Algorithme 4 : Algorithme de programmation dynamique pour le partage du trésor

Entrées : P : liste de pièces T, n : entiers**Sorties** : R : liste de pièces**1 Fonction** *partageBool*($P, \cdot$) :**Données** : $R[T]$: tableau de booléens, **init** à *false***2 début****3** **si** $T \% 2 \neq 0$ **alors retourner** *false***4** $R[0] \leftarrow true$ **5** $obj \leftarrow T/2$ **6** $sum \leftarrow 0$ **7** **pour** $p \leftarrow 1$ à n **faire****8** $sum \leftarrow sum + val(p)$ **9** **pour** $j \leftarrow p$ à sum **faire** $R[j] \leftarrow R[j] \vee R[j - val(p)]$ **10** **retourner** $R[obj]$

l'algorithme 4 n'évalue pas plusieurs fois la même solution. Sa complexité en calculs est cependant en $O(n^2)$, ce qui est assez limité. Ici c'est la complexité en mémoire qui permet de baisser la complexité des calculs. Le problème est ainsi NP-Complet au sens faible.

Solution question 3.4 : *Comment modifier la fonction partageDyn pour obtenir une liste des pièces qui représente la moitié du trésor ?*

Pour obtenir la liste des pièces qui représente la moitié du trésor, il faut ajouter la génération de listes de pièces pour chacune des valeurs objectif. L'algorithme 5 est une version modifiée de l'algorithme 5 qui retourne une liste vide s'il n'y a pas de solution ou une liste des pièces.

Algorithme 5 : Algorithme de programmation dynamique pour le partage du trésor

Entrées : P : liste de pièces

T, n : entiers

Sorties : R : liste de pièces

1 Fonction *partageBool*(P, obj) :

Données : $R[T]$: tableau de listes, **init** à $\emptyset$

2 début

3 **si** $T \% 2 \neq 0$ **alors retourner** $\emptyset$

4 $R[0] \leftarrow \emptyset$

5 $obj \leftarrow T/2$

6 $sum \leftarrow 0$

7 **pour** $p \leftarrow 1$ **à** n **faire**

8 $sum \leftarrow sum + val(p)$

9 **pour** $j \leftarrow p$ **à** sum **faire** **si** $(R[j] = \emptyset) \wedge (R[j - val(p)] \neq \emptyset)$ **alors**

10 $R[j] \leftarrow R[j - val(p)]$

11 $R[j].add(i)$

12

13 **retourner** $R[obj]$
